



Formalar va jinja template tushuncha Foydalanuvchi formani to'ldirdi → server qabul qildi → database'ga yozildi → keyin database'dan qayta o'qib browserda ko'rsatildi

1. Formalar

HTML form foydalanuvchidan ma'lumot oladi:

```
<form method="POST">
    {% csrf_token %}

    <input type="text" name="username">
    <input type="email" name="email">

    <button type="submit">Register</button>
</form>
```

Django `views.py` da:

```
def main(request):
    if request.method == "POST":
        username = request.POST.get("username")
```

```

email = request.POST.get("email")

print(username)
print(email)

return render(request, "index.html")

```

Ya'ni:

```

HTML form
  ↓
POST
  ↓
request.POST
  ↓
View
  ↓
Model / Database

```

Keyinchalik **Django Forms** (`forms.py`) ishlatish mumkin. U validationni ancha qulay qiladi.

2. Jinja / Django Template

Sening hozirgi koding aynan template syntax'dan foydalanmoqda:

```

{% for d in deps %}
    <tr>
        <td>{{ d.username }}</td>
    </tr>
{% endfor %}

```

Bu yerda:

`{{ }}` → **ma'lumotni chiqarish**

```

{{ d.username }}

```

`{% %}` → **logic**

```
{% for d in deps %}
{% endfor %}
```

Masalan:

```
{% if user.is_authenticated %}
    Hello, {{ user.username }}
{% else %}
    Please login
{% endif %}
```

Yana:

```
{% for d in deps %}
    <p>{{ d.username }}</p>
{% empty %}
    <p>No users found</p>
{% endfor %}
```

3. Sening hozirgi kodingda

View:

```
queryset=Person.objects.all()context= {"deps":queryset}
returnrender(request,"dara.html",context)
```

Template:

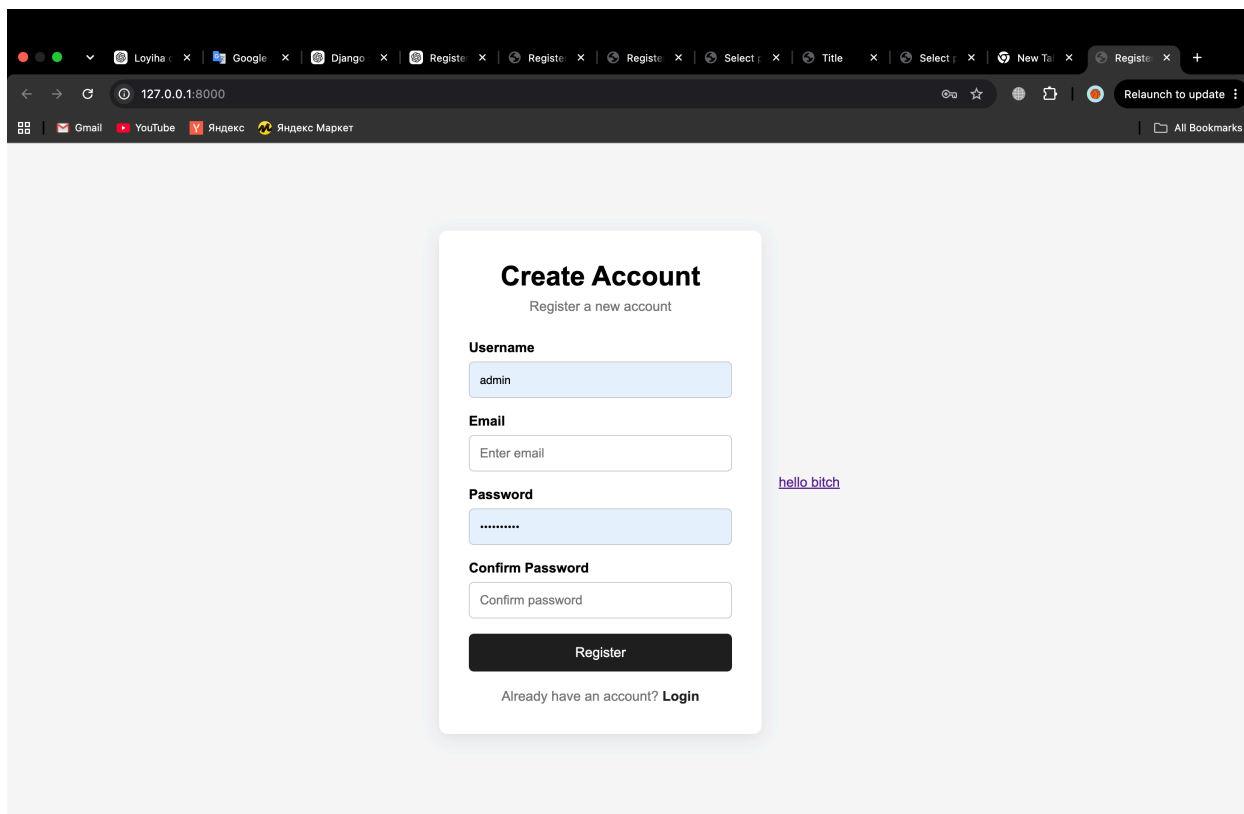
```
{% for d in deps %}
    {{ d.username }}
{% endfor %}
```

Bu juda muhim bog'lanish:

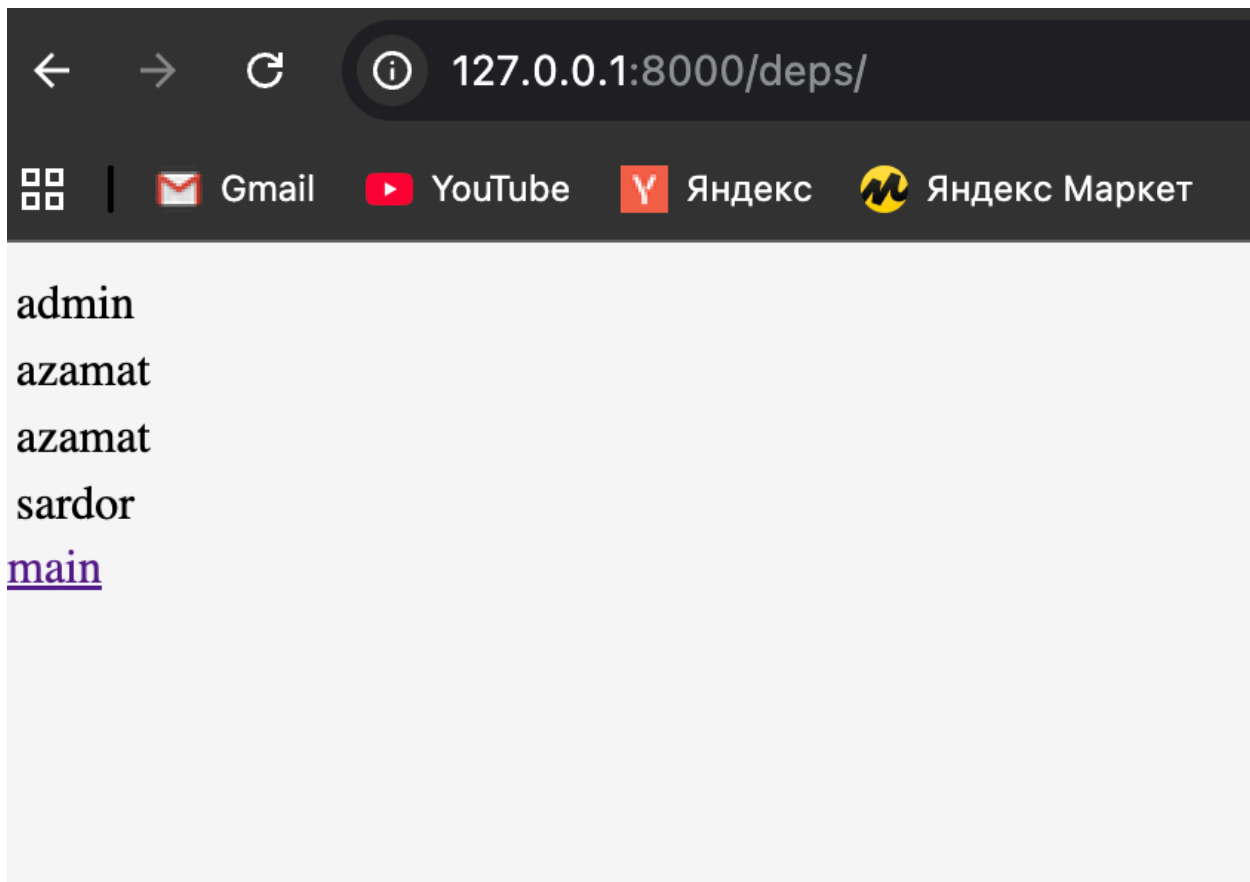
```
context["deps"]  
  ↓  
  deps  
  ↓  
{% for d in deps %}  
  ↓  
  d.username
```

Qisqasi:

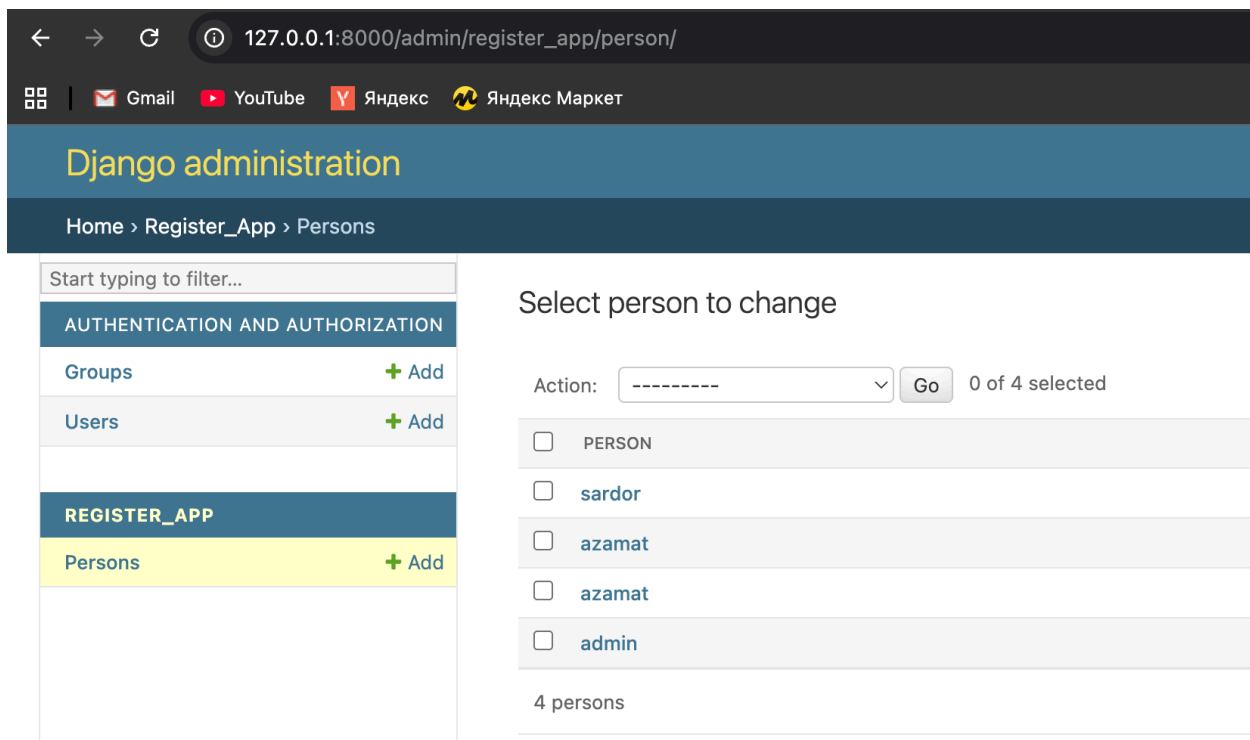
- `forms.py` → formalarni boshqarish va validation
- `request.POST` → yuborilgan ma'lumotni olish
- `{{ variable }}` → ma'lumotni HTML'ga chiqarish
- `{% for %}` → loop
- `{% if %}` → condition
- `{% csrf_token %}` → POST formalar uchun CSRF himoyasi
- `{% load static %}` → CSS/JS/image kabi static fayllarni ulash



hello bitchni bossa, kimlar ruyhatdan
utkanini tuliq ruyhati keladi.



va bular hammasi django/admin paneldagi bazada saqlanadi



Shu loyihani 0 dan quramiz

venv o'rnatamiz

```
python -m venv venv
```

```
source venv/bin/activate
```

django urnatamiz

```
pip install django
```

django da proect ochib olamiz

```
django-admin startproject config .
```

migratsiya qilib db ochamiz

```
python manage.py makemigrations
```

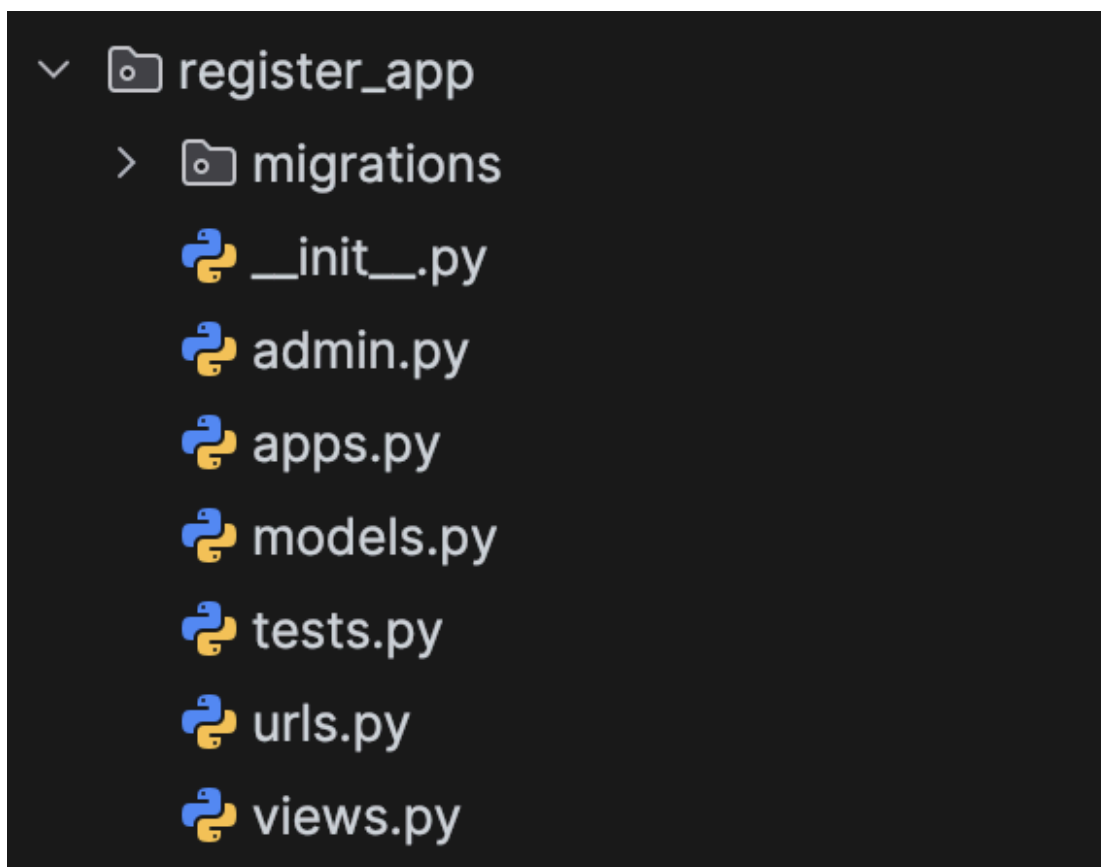
```
python manage.py migrate
```

endi admin/ panelga kirish uchun, super user yaratamiz

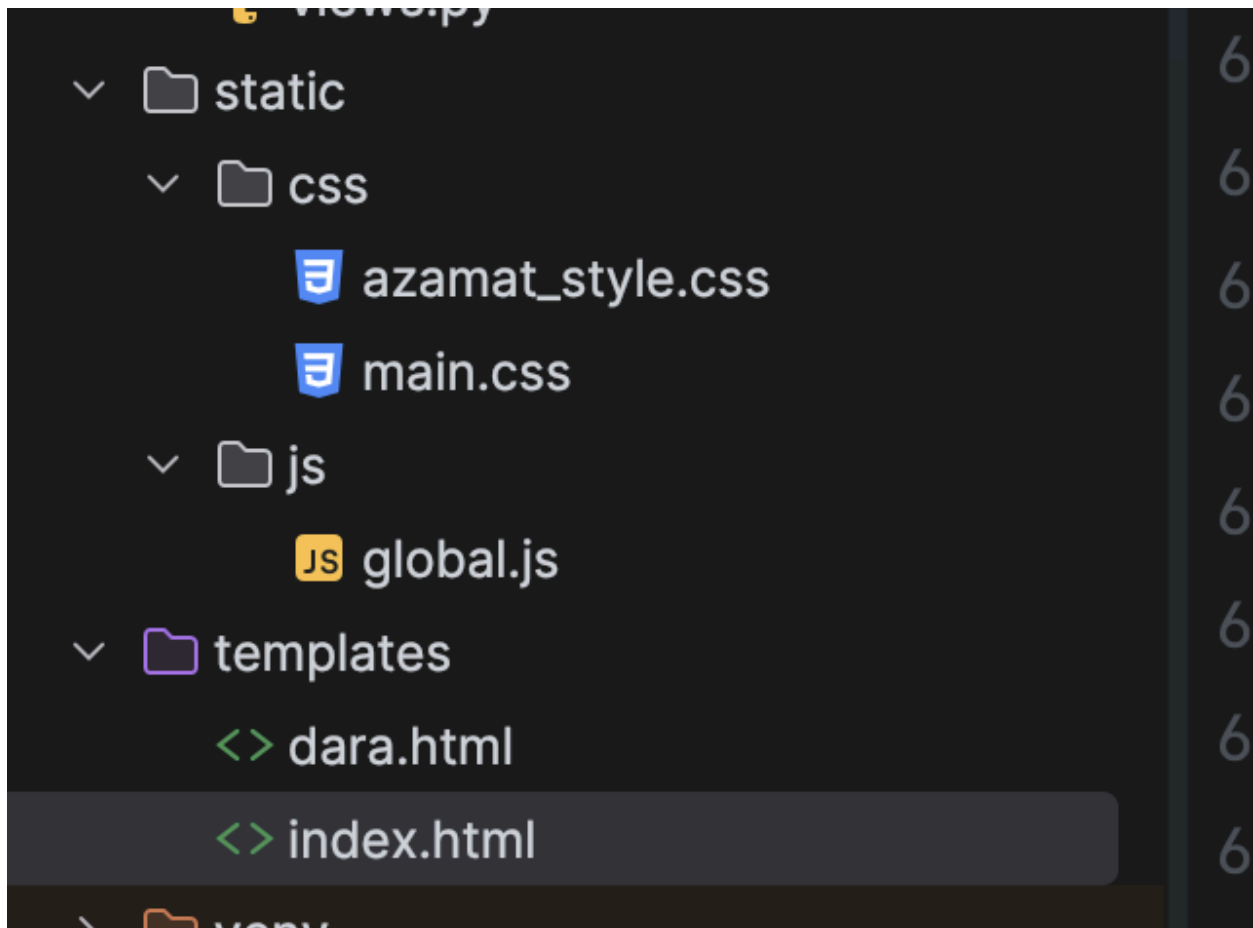
```
python manage.py createsuperuser
```

endi loyihamiz uchun app yaratamiz

```
python manage.py startapp register_app
```



app ni ichiga uzimiz qulda urls.py yaratamiz



config/settings.py

```
"""
Django settings for config project.

Generated by 'django-admin startproject' using Django 5.2.17.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/

```

```

"""
import os
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdi
r'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for productio
n
# See https://docs.djangoproject.com/en/5.2/howto/deployment/
checklist/

# SECURITY WARNING: keep the secret key used in production se
cret!
SECRET_KEY = 'django-insecure-l0#if@)se@o()y7mkt67cz=wt^wod0%
f@n1l=!2-^tr*_k!l6f'

# SECURITY WARNING: don't run with debug turned on in product
ion!
DEBUG = True

ALLOWED_HOSTS = ["*"]

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'register_app',

```

```

]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'config.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR.joinpath("templates")],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'config.wsgi.application'

```

```

# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

```

```

# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True


# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = 'static/'
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'static')
]


# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

```

config/urls.py

```

"""
URL configuration for config project.

The `urlpatterns` list routes URLs to views. For more informa
tion please see:
    https://docs.djangoproject.com/en/5.2/topics/http/urls/

```

Examples:

Function views

1. Add an import: `from my_app import views`
2. Add a URL to urlpatterns: `path('', views.home, name='home')`

Class-based views

1. Add an import: `from other_app.views import Home`
2. Add a URL to urlpatterns: `path('', Home.as_view(), name='home')`

Including another URLconf

1. Import the `include()` function: `from django.urls import include, path`
2. Add a URL to urlpatterns: `path('blog/', include('blog.urls'))`

"""

```
from django.contrib import admin
from django.urls import path, include
```

```
import register_app
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('register_app.urls')),
]
```

register_app/admin.py

```
from django.contrib import admin
from .models import Person
admin.site.register(Person)
# Register your models here.
```

register_app/models.py

```

from django.db import models

# Create your models here.
class Person(models.Model):
    username = models.CharField(max_length=150)
    email = models.EmailField(max_length=250)
    password = models.CharField(max_length=150)
    confirmpassword = models.CharField(max_length=150)

    def __str__(self):
        return f"{self.username}"

```

register_app/urls.py

```

from django.urls import path
from .views import main, get_information

urlpatterns = [
    path('', main, name="main"),
    path('deps/', get_information, name="deps-list")
]

```

register_app/views.py

```

from django.shortcuts import render
from .models import Person
from django.http import HttpResponse
# Create your views here.

def main(request):
    if request.POST:
        model = Person()

```

```

        model.username = request.POST.get('username', '')
        model.email = request.POST.get('email', '')
        model.password = request.POST.get('password', '')
        model.confirmpassword = request.POST.get('confirmpass
word', '')
        model.save()
        print(request.POST)

    return render(request, 'index.html')

def get_information(request):
    queryset = Person.objects.all()
    context = {"deps": queryset}
    return render(request, "dara.html", context)

```

static/css/azamat_style.css

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background: #f4f6f8;

    min-height: 100vh;

    display: flex;
    justify-content: center;
    align-items: center;
}

```

```
.register-container {
  width: 100%;
  max-width: 420px;
  padding: 20px;
}

.register-form {
  background: white;
  padding: 35px;

  border-radius: 10px;

  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.08);
}

h1 {
  text-align: center;
  margin-bottom: 8px;
}

.subtitle {
  text-align: center;
  color: #777;
  margin-bottom: 30px;
}

.form-group {
  margin-bottom: 18px;
}

label {
  display: block;
  margin-bottom: 7px;
  font-weight: bold;
}
```

```
input {
  width: 100%;
  padding: 12px;

  border: 1px solid #ccc;
  border-radius: 6px;

  font-size: 15px;
}

input:focus {
  outline: none;
  border-color: #333;
}

button {
  width: 100%;
  padding: 13px;

  background: #222;
  color: white;

  border: none;
  border-radius: 6px;

  font-size: 16px;
  cursor: pointer;
}

button:hover {
  background: #000;
}

.error {
  color: red;
  font-size: 14px;
}
```

```

    margin-bottom: 15px;
}

.login-text {
    text-align: center;
    margin-top: 20px;
    color: #777;
}

.login-text a {
    color: #222;
    text-decoration: none;
    font-weight: bold;
}

```

static/js/global.js

```

document.addEventListener("DOMContentLoaded", function () {

    const form = document.getElementById("registerForm");
    const errorMessage = document.getElementById("errorMessage");

    if (!form) {
        return;
    }

    form.addEventListener("submit", function (event) {

        const username = document.getElementById("username").value.trim();
        const email = document.getElementById("email").value.trim();
    });

```

```

        const password = document.getElementById("password").
value;
        const confirmPassword = document.getElementById("conf
irmPassword").value;

        // Clear previous error
        errorMessage.textContent = "";

        // Check username
        if (!username) {
            errorMessage.textContent = "Please enter your use
rname.";
            return;
        }

        // Check email
        if (!email) {
            errorMessage.textContent = "Please enter your ema
il.";
            return;
        }

        // Check password
        if (!password) {
            errorMessage.textContent = "Please enter your pas
sword.";
            return;
        }

        // Check password confirmation
        if (password !== confirmPassword) {
            errorMessage.textContent = "Passwords do not matc
h!";
            return;
        }

```

```

        console.log("Username:", username);
        console.log("Email:", email);

        alert("Registration successful!");
    });
});

```

templates/index.html

```

<!DOCTYPE html>
<html lang="en">
{% load static %}
<head>
    <link rel="stylesheet" href="{% static 'css/azamat_style.
css' %}">

    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initia
l-scale=1.0">
    <title>Register</title>

    <title>Title</title>
<body>

    <div class="register-container">

        <form method="POST" id="registerForm" class="register
-form">

            {% csrf_token %}
            <h1>Create Account</h1>
            <p class="subtitle">Register a new account</p>

            <div class="form-group">
                <label for="username">Username</label>

```

```

        <input
            type="text"
            id="username"
            name="username"
            placeholder="Enter username"
            required
        >
    </div>

    <div class="form-group">
        <label for="email">Email</label>
        <input
            type="email"
            id="email"
            name="email"
            placeholder="Enter email"
            required
        >
    </div>

    <div class="form-group">
        <label for="password">Password</label>
        <input
            type="password"
            id="password"
            name="password"
            placeholder="Enter password"
            required
        >
    </div>

    <div class="form-group">
        <label for="confirmPassword">Confirm Password
    </label>
        <input
            type="password"

```

```

        id="confirmPassword"
        name="confirmpassword"
        placeholder="Confirm password"
        required
    >
</div>

<p id="errorMessage" class="error"></p>

<button type="submit">Register</button>

<p class="login-text">
    Already have an account?
    <a href="#">Login</a>
</p>

</form>

</div>
<a href="{% url 'deps-list' %}"> hello bitch </a>

<script src="{% static 'js/global.js' %}"></script>
</body>
</html>

```

templates/dara.html

```

<!DOCTYPE html>
<html lang="en">
{% load static %}
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>

```

```
<body style="background-color:whitesmoke">

<table>
  <tbody>
    {% for d in deps %}
    <tr>
      <td>{{d.username}}</td>

    </tr>
    {% endfor %}

  </tbody>
</table>

<a href="/" class="button">main</a>
</body>
</html>
```